

美股分析系统 - 开发日志与技术文档

文档版本: v1.0

创建日期: 2026-05-08

最后更新: 2026-05-08

目录

- [项目概述](#)
- [技术架构](#)
- [技术选型与决策](#)
- [核心功能实现](#)
- [数据库设计](#)
- [部署配置](#)
- [关键问题与解决方案](#)
- [未来扩展方向](#)

项目概述

基本信息

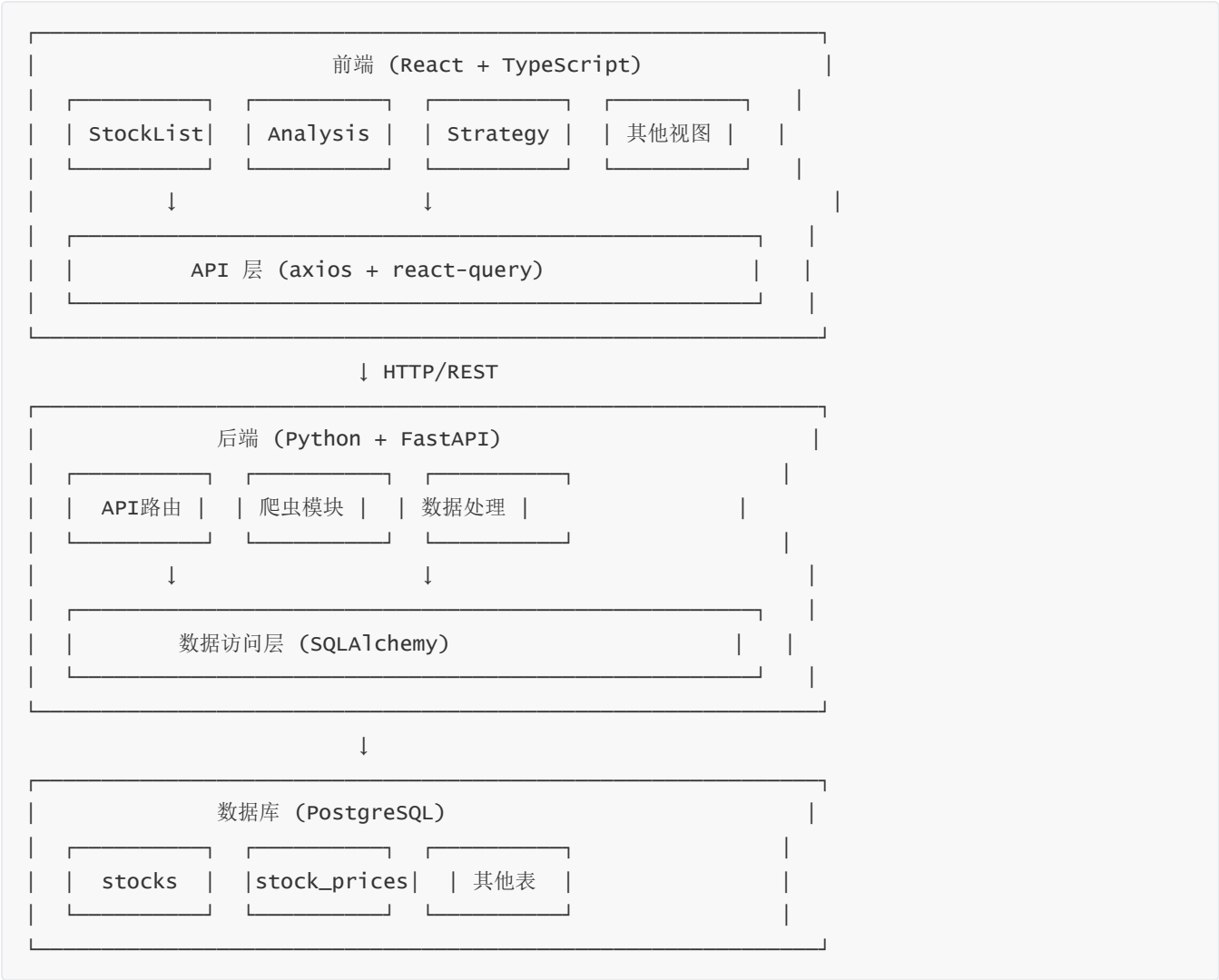
- 项目名称:** 美股数据分析系统 (014_stock-us)
- 项目类型:** 全栈 Web 应用
- 主要功能:** 美股数据展示、K线图分析、技术指标计算
- 目标用户:** 美股投资者、量化分析爱好者
- 数据源:** Yahoo Finance (通过 yfinance 库)

项目目标

- 提供主要美股的实时/历史数据展示
- 实现交互式 K 线图分析
- 支持技术指标计算与展示
- 提供简洁现代的用户界面

技术架构

整体架构图



技术栈详解

前端技术栈

技术	版本	用途	选型理由
React	18+	UI 框架	组件化、生态丰富
TypeScript	5+	类型安全	减少运行时错误
Vite	5+	构建工具	快速热更新
TDesign	latest	UI 组件库	腾讯设计系统、企业级
ApexCharts	3.4+	图表库	轻量、现代、易用
React Router	6+	路由管理	标准路由方案
i18next	23+	国际化	支持中英文切换

技术	版本	用途	选型理由
axios	1.6+	HTTP 客户端	标准 HTTP 库
react-query	3.39+	数据请求	缓存、自动刷新

后端技术栈

技术	版本	用途	选型理由
Python	3.11+	编程语言	数据处理生态丰富
FastAPI	0.109+	Web 框架	高性能、自动 API 文档
SQLAlchemy	2.0+	ORM	灵活、强大
yfinance	0.2+	数据源	免费、易用
pandas	2.1+	数据处理	标准数据处理库
PostgreSQL	15+	数据库	可靠、功能强大
psycopg2	2.9+	数据库驱动	PostgreSQL 标准驱动

部署技术栈

技术	用途
Docker	容器化
Docker Compose	多容器编排
Nginx	前端静态文件服务、反向代理
Git	版本控制
post-receive hook	自动部署

技术选型与决策

1. 图表库选型：lightweight-charts → ApexCharts

问题背景

- 初始选型使用 `lightweight-charts` v4.2.3
- 遇到 API 不兼容问题: `chart.addCandlestickSeries` is not a function
- v4.x 版本 API 发生重大变更，旧代码无法运行

备选方案对比

方案	优点	缺点	包大小
ECharts	功能丰富、文档完善	包体积大、偏商务风格	~1.2MB
ApexCharts	现代简洁、易用、轻量	定制化能力稍弱	~200KB
lightweight-charts	专业金融图表	API 不稳定、文档少	~300KB

最终决策

选用 ApexCharts，理由：

- 1. ☒ 包体积小（~200KB），加载快
- 2. ☒ API 简洁，学习成本低
- 3. ☒ 默认样式现代美观
- 4. ☒ React 集成良好（`react-apexcharts`）
- 5. ☒ 维护活跃，API 稳定

implementation 要点

- 在 `StockList.tsx` 中使用 `react-apexcharts` 组件
- 配置 `candlestick` 图表类型
- 处理日期数据（整数时间戳）
- 自定义颜色方案（涨跌颜色）

2. 日期数据处理方案

问题背景

- 前后端数据交换需要统一的日期格式
- 避免时区问题
- 确保跨平台兼容性

方案对比

方案	优点	缺点
ISO 8601 字符串	可读性好、标准格式	时区处理复杂
Date 对象	JavaScript 原生支持	序列化问题
整数时间戳	简单、跨平台、无时区问题	可读性差

最终决策

使用整数时间戳 (Unix timestamp) , 理由:

- 1. ☒ 跨平台兼容 (Python、JavaScript、数据库)
- 2. ☒ 无时区问题 (使用 UTC)
- 3. ☒ 序列化简单 (JSON 原生支持数字)
- 4. ☒ 比较和计算方便

实现要点

- 后端: 使用 `datetime.timestamp()` 生成时间戳
- 前端: 使用 `new Date(timestamp * 1000)` 转换
- 数据库: 存储为 `BIGINT` 类型

3. 股票列表设计

设计目标

- 覆盖主要美股
- 平衡数据量 (下载速度 vs 覆盖范围)
- 包含不同类别 (大盘股、中概股、ETF)

当前方案: 硬编码精选列表

股票组成 (共 81 只) :

类别	数量	示例	说明
S&P 500 头部	~60	AAPL, MSFT, GOOGL, NVDA, META, TSLA	美国大盘龙头
中概股	8	BABA, JD, PDD, BIDU, NIO, LI, XPEV, TCOM	中国概念股
ETF	9	SPY, QQQ, DIA, VTI, VOO, GLD, SLV	指数基金、商品 ETF
国际头部	7	TSM, ASML, SAP, NVS, NVO, TM, SONY	非美国龙头

代码位置: `backend/crawlers/us_stock_source.py` 第 17-27 行

```
MAJOR_US_STOCKS = [  
    "AAPL", "MSFT", "GOOGL", "AMZN", "NVDA", "META", "BRK-B", "TSLA",  
    "JPM", "V", "UNH", "XOM", "PG", "JNJ", "MA", "HD", "CVX", "MRK",  
    # ... 完整列表见源代码  
]
```

设计考量

优点:

1. ☒ 数据下载速度快 (81 只 × 10 年历史)
2. ☒ 存储占用小
3. ☒ 覆盖主要行业和美国大盘
4. ☒ 简单易维护

缺点:

1. ☒ 不包含小盘股和微盘股
2. ☒ 硬编码, 扩展需要修改代码
3. ☒ 股票代码变更时需要手动更新

扩展方案

方案 1: 手动添加

- 直接在 `MAJOR_US_STOCKS` 列表中添加

方案 2: 动态获取

- 从指数成分股列表获取 (S&P 500、NASDAQ 100)
- 需要修改爬虫逻辑
- 考虑数据下载时间和存储压力

方案 3: 用户自定义

- 数据库支持动态添加股票
- 前端提供股票搜索和添加功能
- 需要修改前端和后端逻辑

4. 国际化方案

技术选型

- 库: `i18next + react-i18next`
- 支持语言: 中文 (默认)、英文

实现要点

- 翻译文件: `frontend/src/i18n/zh.ts`、`frontend/src/i18n/en.ts`
- 配置文件: `frontend/src/i18n/index.ts`
- 使用方式: `t('key')` 获取翻译

文件结构

```
frontend/src/i18n/
├── index.ts    # i18n 配置
├── zh.ts      # 中文翻译
└── en.ts      # 英文翻译
```

核心功能实现

1. K 线图功能

组件结构

```
StockList.tsx
├── 股票列表表格
│   ├── 搜索框
│   ├── 表格（可滚动）
│   └── 选中高亮
└── K 线图面板
    ├── 时间范围选择
    └── ApexCharts 图表
```

数据流

```
用户选择股票 → 前端状态更新 → 请求后端 API
→ yfinance 获取数据 → 返回 JSON → 前端渲染图表
```

关键代码位置

- 前端: `frontend/src/views/StockList.tsx`
- 后端 API: `backend/main.py` (API 路由)
- 数据获取: `backend/crawlers/us_stock_source.py`

2. 股票列表功能

功能点

1. ☒ 显示股票列表（代码、名称、价格、涨跌幅）
2. ☒ 搜索过滤
3. ☒ 选中高亮
4. ☒ 点击选择
5. ☒ 表头固定（可滚动）

实现细节

- 表格组件:** TDesign `Table` 组件
- 样式:** 自定义 CSS 变量
- 状态管理:** React `useState`

3. 布局与样式

设计原则

- 简洁现代
- 响应式布局
- 一致的视觉风格

样式方案

- CSS 变量:** 定义颜色、间距等
- SCSS:** 使用 SCSS 编写样式
- TDesign 主题:** 基于 TDesign 定制

关键样式文件

- `frontend/src/styles/index.scss` - 全局样式
- 各组件的 `inline` 样式 - 特定调整

数据库设计

表结构

stocks 表（股票基本信息）

字段	类型	说明
symbol	VARCHAR(10)	股票代码（主键）
name	VARCHAR(255)	股票名称
sector	VARCHAR(100)	行业分类

stock_prices 表（历史价格数据）

字段	类型	说明
symbol	VARCHAR(10)	股票代码（外键）
date	DATE	日期
open	DECIMAL(10,2)	开盘价

字段	类型	说明
high	DECIMAL(10,2)	最高价
low	DECIMAL(10,2)	最低价
close	DECIMAL(10,2)	收盘价
volume	BIGINT	成交量

索引:

- 主键: (symbol, date)
- 索引: symbol (加速查询)

数据更新策略

当前方案

- 手动触发爬取
- 全量更新 (81 只股票的所有历史数据)

优化方向

- 增量更新 (只获取最新数据)
- 定时自动爬取
- 错误重试机制

部署配置

Docker 配置

Dockerfile (后端)

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerfile.frontend (前端)

```
# 构建阶段
FROM node:18-alpine as builder
WORKDIR /app
COPY frontend/ .
RUN npm install && npm run build

# 生产阶段
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf
```

docker-compose.yml

```
version: '3.8'
services:
  frontend:
    image: nginx:alpine
    volumes:
      - /opt/stock-us-code/frontend:/usr/share/nginx/html
    ports:
      - "8081:80"
    depends_on:
      - backend

  backend:
    build:
      context: .
      dockerfile: Dockerfile
    volumes:
      - /opt/stock-us-code/backend:/app
      - ./data:/app/data
    ports:
      - "8000:8000"
    depends_on:
      - db

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: stocks
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

自动部署配置

Git 部署流程

本地开发 → git push → 服务器接收 → post-receive hook → 自动部署

post-receive hook 配置

文件位置: `/opt/stock-us-code/hooks/post-receive`

内容:

```
#!/bin/bash
git --work-tree=/opt/stock-us-code/checkout --git-dir=/opt/stock-us-code checkout -f
```

作用:

- 接收 push 的代码
- 自动 checkout 到指定目录
- Docker 容器挂载该目录, 代码自动更新

部署步骤

1. 首次部署:

```
docker-compose up -d --build
```

2. 后续更新:

```
git push production main
# 完成! 代码自动更新
```

3. 后端代码更新:

```
docker-compose restart backend
```

关键问题与解决方案

1. 图表库 API 不兼容

问题: `lightweight-charts` v4.2.3 API 变更导致 `chart.addCandlestickSeries` is not a function

解决方案: 迁移到 `ApexCharts`

经验: 选择维护活跃、API 稳定的库

2. TypeScript 配置错误

问题: `tsconfig.node.json` 配置错误 (`may not disable emit`、`must have setting "composite": true`)

解决方案:

- 添加 `composite: true`
- 移除 `noEmit: true`

经验: TypeScript 配置文件需要仔细检查

3. 表格表头固定

问题: 表格内容过多时, 表头滚动消失

解决方案: 添加 `maxHeight` 限制表格高度

代码:

```
<div className="table-scroll" style={{ maxHeight: 'calc(100vh - 200px)' }}>
```

4. 股票列表选中高亮

问题: 需要视觉反馈显示当前选中的股票

解决方案: 使用 CSS 变量定义选中颜色, 条件渲染样式

代码:

```
<td style={{ color: selected === s.symbol ? 'var(--selected)' : 'var(--accent)' }}>
```

5. 样式不统一

问题: 各组件的样式不一致

解决方案:

1. 定义统一的 CSS 变量
 2. 创建可复用的样式类
 3. 统一字体大小、间距等
-

未来扩展方向

1. 功能扩展

技术指标

- ☐ 移动平均线 (MA)
- ☐ 指数移动平均线 (EMA)
- ☐ 布林带 (BOLL)
- ☐ MACD
- ☐ RSI
- ☐ KDJ

实时数据

- ☐ WebSocket 推送
- ☐ 实时价格更新
- ☐ 实时K线图

策略回测

- ☐ 策略编辑器
- ☐ 回测引擎
- ☐ 性能分析

告警功能

- ☐ 价格告警
- ☐ 指标告警
- ☐ 邮件/短信通知

2. 性能优化

前端优化

- ☐ 虚拟滚动 (长列表)
- ☐ 代码分割
- ☐ 懒加载
- ☐ 缓存策略

后端优化

- ☐ 数据库索引优化
- ☐ 查询优化
- ☐ 缓存层 (Redis)

☐ 异步任务 (Celery)

3. 数据扩展

股票池扩展

- ☐ 支持全市场股票 (6000+)
- ☐ 动态股票列表管理
- ☐ 股票搜索和添加

数据源扩展

- ☐ 多数据源支持
- ☐ 数据验证和清洗
- ☐ 数据更新调度

4. 用户体验

界面优化

- ☐ 深色模式
- ☐ 自定义布局
- ☐ 快捷键支持

交互优化

- ☐ 图表缩放和拖动
- ☐ 多周期切换
- ☐ 指标叠加

附录

A. 文件清单

前端文件

```
frontend/
├─ src/
│   ├─ App.tsx           # 主应用组件
│   ├─ main.tsx          # 入口文件
│   ├─ vite-env.d.ts     # Vite 类型声明
│   ├─ api/
│   │   └─ index.ts      # API 请求封装
│   ├─ i18n/
│   │   ├─ index.ts      # i18n 配置
│   │   └─ zh.ts         # 中文翻译
```

```
| | └─ en.ts          # 英文翻译
| └─ styles/
|   └─ index.scss     # 全局样式
| └─ views/
|   └─ StockList.tsx  # 股票列表页面
|   └─ AnalysisView.tsx # 分析页面
|   └─ StrategyView.tsx # 策略页面
└─ package.json
└─ tsconfig.json
└─ tsconfig.node.json
└─ vite.config.ts
└─ start.bat
```

后端文件

```
backend/
└─ main.py          # FastAPI 应用入口
└─ crawlers/
|   └─ us_stock_source.py # 美股数据源
└─ models.py        # 数据库模型
└─ database.py       # 数据库连接
└─ config.py         # 配置管理
└─ requirements.txt  # Python 依赖
```

配置文件

```
└─ docker-compose.yml # Docker Compose 配置
└─ Dockerfile          # 后端 Dockerfile
└─ Dockerfile.frontend # 前端 Dockerfile
└─ nginx.conf          # Nginx 配置
└─ config.yaml         # 应用配置
└─ requirements.txt     # Python 依赖（根目录）
```

B. 常用命令

开发命令

```
# 前端开发
cd frontend
npm install
npm run dev

# 后端开发
cd backend
pip install -r requirements.txt
uvicorn main:app --reload
```

Docker 命令

```
# 构建并启动
docker-compose up -d --build

# 查看日志
docker-compose logs -f

# 重启服务
docker-compose restart backend

# 停止服务
docker-compose down
```

部署命令

```
# 推送代码
git add .
git commit -m "Update"
git push production main

# 手动重启（如需）
ssh user@server
cd /opt/stock-us-code/checkout
docker-compose restart backend
```

C. 参考资料

官方文档

- [React](#)
- [TypeScript](#)
- [Vite](#)
- [TDesign](#)
- [ApexCharts](#)
- [FastAPI](#)
- [yfinance](#)

教程和指南

- [React + TypeScript 最佳实践](#)
 - [FastAPI 入门教程](#)
 - [ApexCharts React 示例](#)
-

文档维护

更新记录

日期	版本	更新内容	作者
2026-05-08	v1.0	初始版本	-

如何贡献

- 1. 发现错误或缺漏，请提出 Issue
- 2. 想要添加内容，请提交 Pull Request
- 3. 保持文档结构一致
- 4. 使用清晰的 Markdown 格式

文档结束

本文档持续更新中，如有问题或建议，欢迎反馈。